

The Shape Between: A Full-Page Illustrated Storybook Template

Character generation, symbolic page composition, and PDF assembly

Daniel Ari Friedman
Active Inference Institute
`daniel@activeinference.institute`
ORCID: 0000-0001-6232-9096

July 3, 2026

Contents

1	Abstract	2
2	Introduction	3
3	Architecture	4
4	Story and Graphics	5
5	Reproducibility	6
6	References	7

1 Abstract

`template_storybook` demonstrates a public, standalone picture-book workflow in the research template repository. The bundled project renders a deterministic nine-page storybook in which a child tetrahedron raised by cubes meets a child cube raised by tetrahedra. The story uses a large reciprocal symbol to frame belonging without sameness: square and triangle families remain distinct while the space between them becomes navigable.

The project separates story data, rendering logic, and orchestration. Story text, characters, page order, palettes, and overlay choices live in `content/story.yaml`; character generation and illustration live in `src/storybook/`; one script renders the cover, each page script renders a numbered page, and a final script assembles the PDF. This makes the exemplar a forkable pattern for full-page creative artifacts rather than standard manuscript-centered figure generation.

2 Introduction

Most template exemplars in this repository produce analytical reports, manuscripts, newspapers, textbooks, or structured research artifacts. A storybook stresses a different surface: the designed page is the artifact. The page must be a complete visual composition, text must remain readable, and the pipeline must still expose deterministic source data, tests, and generated outputs.

The bundled story is intentionally symbolic. Tessa is a tetrahedron in a family of cubes. Ciro is a cube in a tetrahedral family. Their meeting is arranged around a yin-yang-like valley so that each family contains a trace of the other without erasing its own geometry. The point is not mathematical instruction; it is a compact visual grammar for reciprocal belonging.

3 Architecture

The storybook uses the same Layer-2 project contract as the other public templates:

- `content/story.yaml` owns cast records and page records.
- `src/storybook/characters.py` validates shape characters.
- `src/storybook/story.py` loads the content file into typed records.
- `src/storybook/illustration.py` renders the full-page scenes.
- `src/storybook/rendering.py` assembles PNG pages into the PDF and writes the manifest.
- `scripts/` contains the thin Stage-02 entry points.

The page scripts are deliberately repetitive. Their job is visible orchestration: cover, page 1, page 2, and so on. The repetition keeps the creative page surface easy to inspect while preventing scripts from becoming hidden rendering engines.

4 Story and Graphics

Every page is rendered as a full-page PNG. Some pages place text directly over the illustration with shadowed lettering; others use a translucent box when the scene is visually dense. The YAML `overlay_box` value controls that choice per page.

The renderer uses simple geometric primitives: isometric cubes, shaded tetrahedra, family clusters, a curved reciprocal symbol, and deterministic star fields. This gives forks a readable baseline. Replacing the story does not require editing Python; replacing the visual grammar does.

5 Reproducibility

The project avoids external image services and live APIs. Page images are procedurally generated from checked-in content, and the final PDF is assembled from those generated images. Tests check that the story loads, the two central characters have opposite family shapes, a page renders at the configured dimensions, the PDF page count matches the content file, and at least one page script can run against a temporary project root.

The generated manifest records the cast, page list, and render paths. That manifest is the compact evidence surface for the primary artifact.

6 References

The project is an illustrative template. The current manuscript does not depend on external scholarly claims.

References